

Computer Networks LAB (CS3P004)

Spring Semester 2024-25

End Sem Lab Project

22CS01076 Borru Vijay Sai

22CS01013 C V Harshith Reddy

22CS01006 N Sai Charan Reddy

Topic: TCP Congestion Control Mechanisms in Wireless Networks: Performance Analysis with High Packet Loss

Project Overview:

In this project analyzes the performance of different TCP congestion control mechanisms (NewReno, Cubic, and Vegas) in wireless networks experiencing high packet loss. TCP congestion control mechanisms regulate data transmission rates to prevent network congestion while maximizing throughput. Through network simulation, we examine how these TCP variants adapt their congestion windows in response to packet loss events and compare their throughput and delay characteristics.

Tools used: NS3 (Network Simulator 3) for simulation and Matplotlib for data visualization.

NS3 Utilization in Project:

NS-3 is a discrete-event network simulator widely used for research and education, providing realistic modeling of complex network scenarios without the need for physical hardware. It supports both wired and wireless network simulations and is programmable in C++ with optional Python bindings. For our project, NS-3 enabled the creation of a wireless topology with high packet loss, where each node operated a different TCP congestion control algorithm (NewReno, Cubic, Vegas). Its tracing framework collected detailed metrics such as congestion window

dynamics, throughput, and delays, which were critical for performance analysis. Visualization tools like matplotlib were used to plot results and compare TCP behaviors. This approach allowed us to directly observe and analyze how each TCP variant responded to challenging wireless conditions within a controlled, reproducible environment.

Project Objectives:

This project aims to:

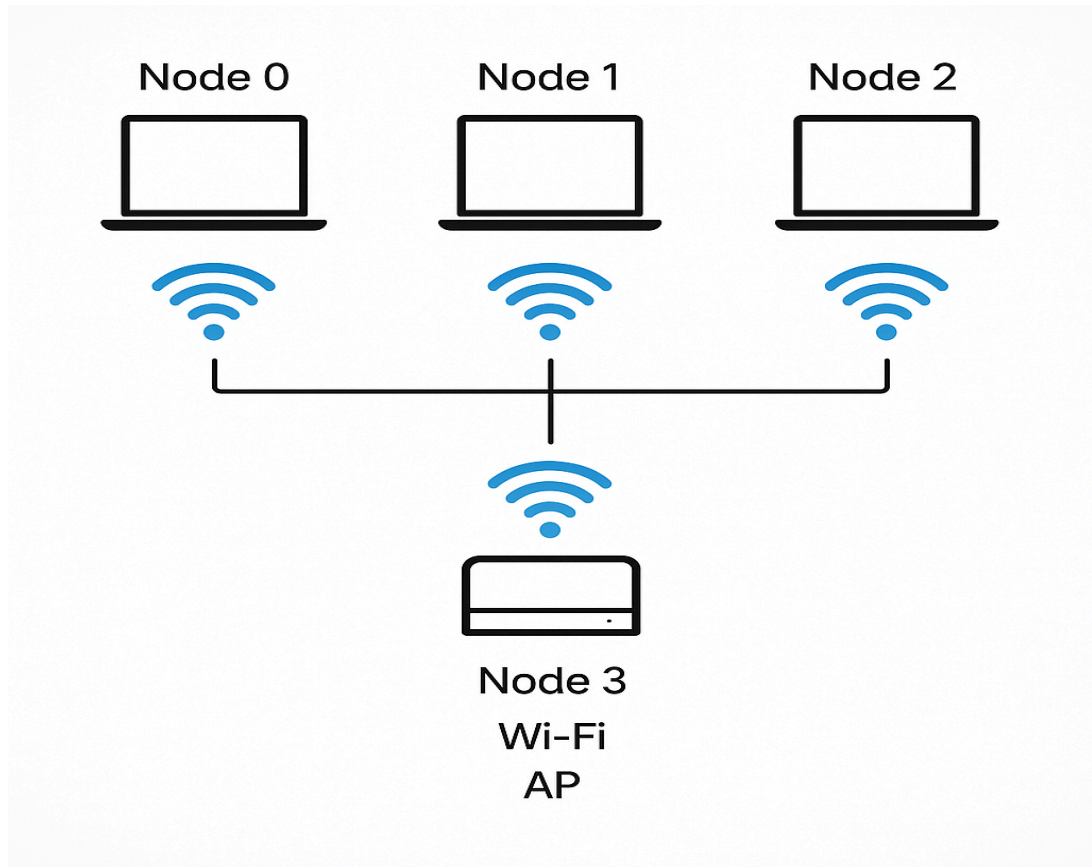
- Compare the performance of NewReno, Cubic, and Vegas TCP variants in wireless networks with high packet loss.
- Analyze congestion window evolution patterns for each algorithm.
- Measure and evaluate throughput, packet loss, and delay metrics.

Implementation:

Network Setup and Topology:

This simulation was conducted using an NS-3 network simulator, creating a wireless topology with multiple nodes implementing different TCP congestion control algorithms. We configured a wireless network with high packet loss to simulate challenging network conditions. The simulation utilized a wireless network configuration with four nodes (10.1.1.1 through 10.1.1.4) in which three are STA (Station) and one is an access point (AP) with node 10.1.1.4 serving as a common destination. Six bidirectional flows were established between nodes:

- Flow 1 & 2: Between nodes 10.1.1.1 (NewReno) and 10.1.1.4
- Flow 3 & 4: Between nodes 10.1.1.2 (Cubic) and 10.1.1.4
- Flow 5 & 6: Between nodes 10.1.1.3 (Vegas) and 10.1.1.4



Experimental Setup:

- **Network Topology:** Three sender nodes connected to a receiver node via wireless links
- **TCP Variants:** NewReno (Node 1), Cubic (Node 2), and Vegas (Node 3)
- **Simulation Duration:** 120 seconds
- **Metrics Collected:** Congestion window size, throughput, packet loss, and delay
- **Packet Loss:** Induced through wireless signal attenuation and interference

Plotting Results:

Finally, Plotting cwnd vs time for all three TCP variants.

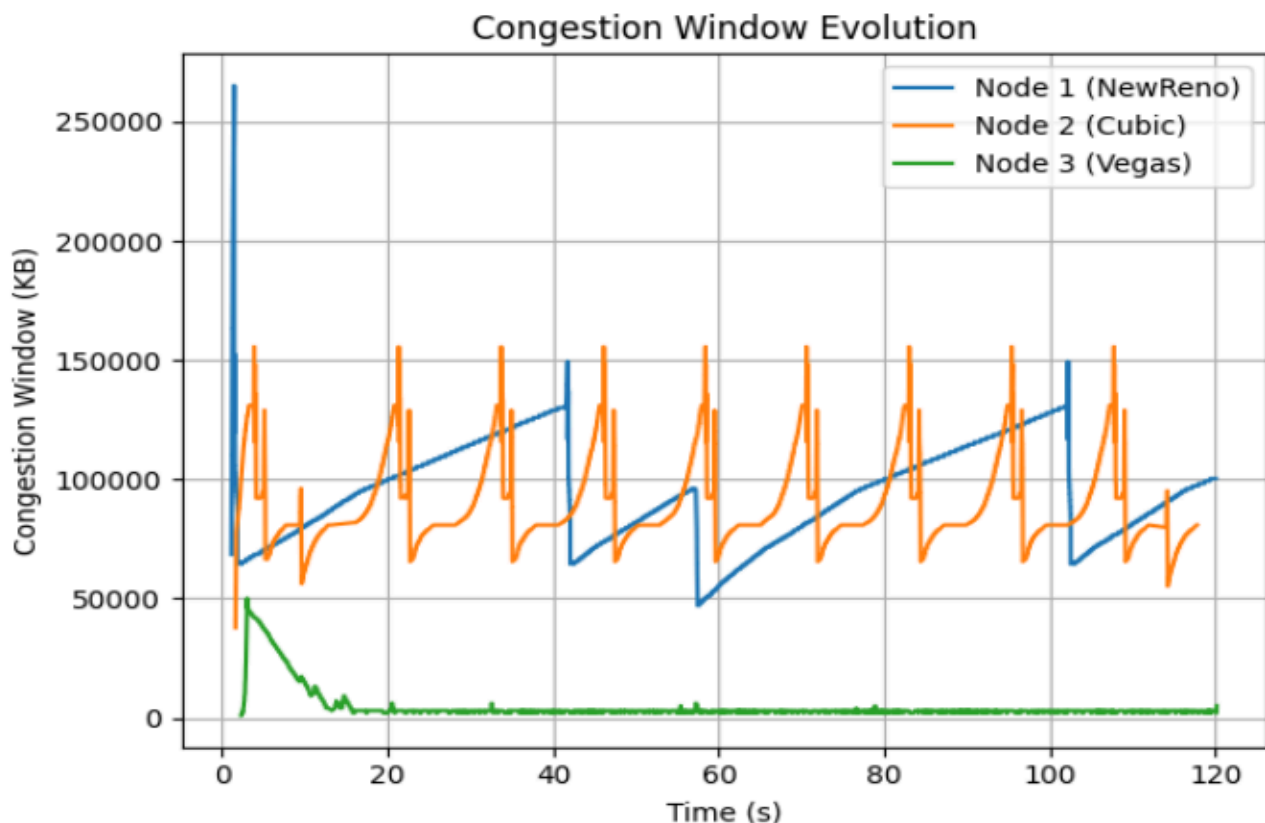
TCP Variants Studied:

TCP NewReno (Node 1): An enhancement of standard TCP Reno that improves recovery during multiple packet losses from a single window of data, featuring a conservative congestion window increase mechanism

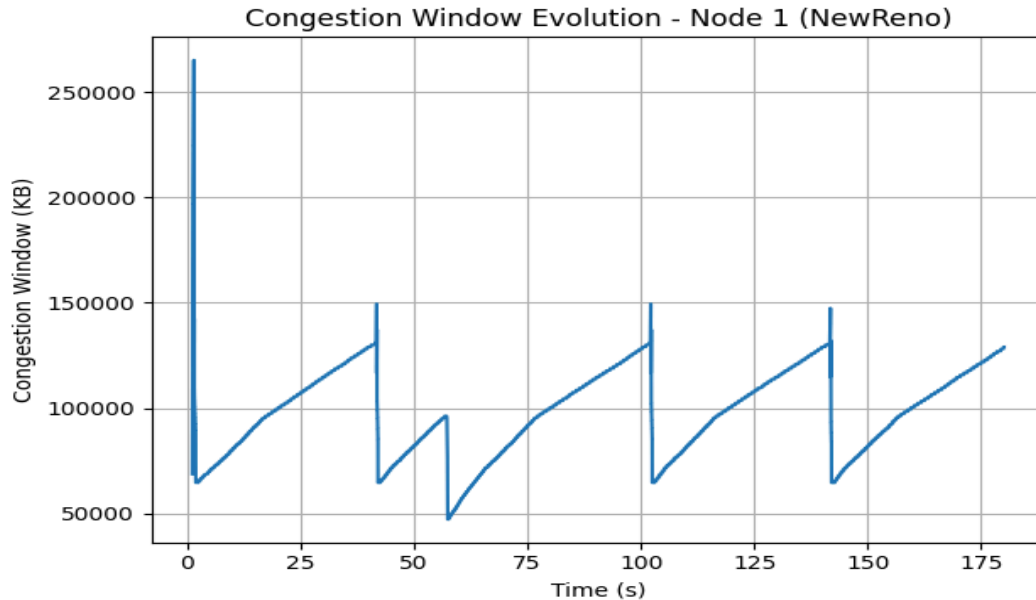
TCP Cubic (Node 2): Designed for high-bandwidth networks with high latency, using a cubic function for window growth that is less aggressive when approaching the previous maximum window size and more aggressive when moving away from it

TCP Vegas (Node 3): A delay-based congestion control algorithm that uses RTT (Round-Trip Time) measurements rather than packet loss to detect congestion, proactively reducing its sending rate before packet loss occurs.

Results and Observations: Congestion Window Behavior-

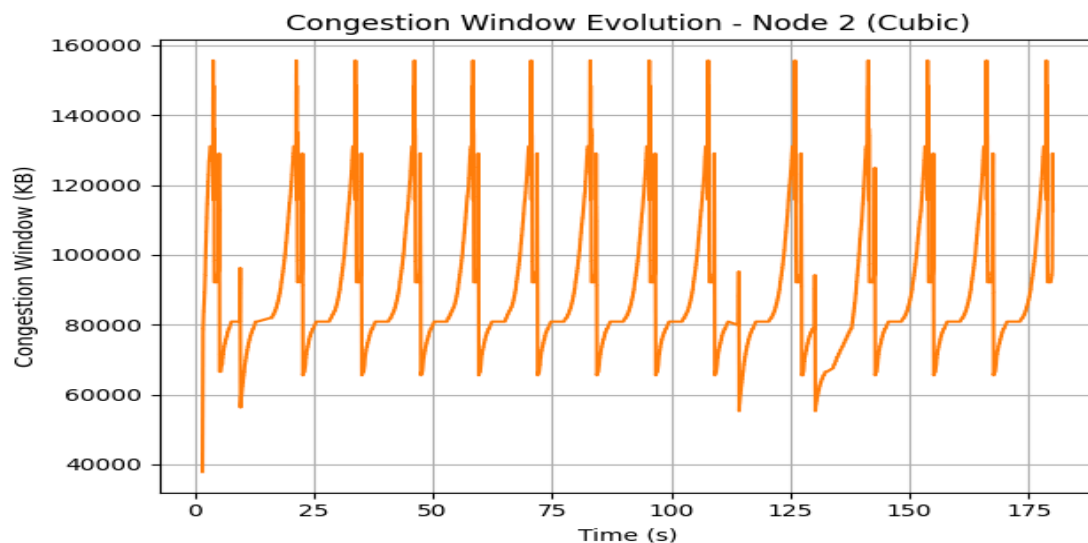


Node 1 (NewReno):



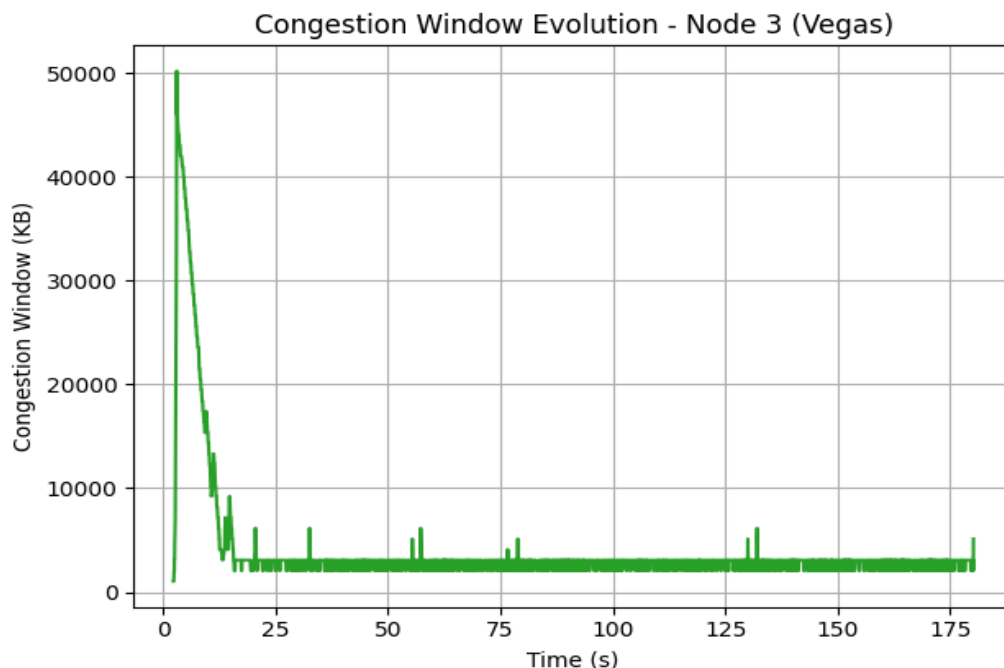
- **Observations:** Classic sawtooth pattern with congestion window reaching ~150KB with steady linear increase in CWND (typical AIMD behavior). Multiple recovery cycles visible throughout simulation.
- **Interpretation:** NewReno is conservative, increases slowly, and responds to packet loss by halving CWND.
- **Implication:** Moderate throughput, moderate delay. Stable behavior but not aggressive.

Node 2 (Cubic):



- **Observations:** Window size peaks around 150KB. Exhibits cubic growth function behavior with rapid bandwidth reclamation. Aggressive growth followed by sharp drops.
- **Interpretation:** Cubic is more aggressive in increasing CWND but backs off sharply on loss.
- **Implication:** Higher throughput potential, but frequent large fluctuations cause higher delay and instability—especially noticeable in lossy wireless environments.

Node 3 (Vegas)



-
- **Observations:** Dramatically different behavior - stabilizes at very low window size (~50KB). Rapid rise initially, then levels off and stays low.
- **Interpretation:** Vegas detects early signs of congestion using RTT and adjusts CWND accordingly before loss happens.
- **Implication:** Low delay, but also lower throughput. It avoids congestion, ideal for delay-sensitive apps.

Throughput Analysis:

The aggregate throughput measurements show:

- Initial ramp-up period (1.1s - 7.1s) as connections establish

- Relatively stable throughput around 13 Mbps during mid-simulation
- Several throughput drops (e.g., at 22s, 42s, 59s, 71s) indicating congestion events
- Recovery periods following throughput drops

```
Flow 1 (10.1.1.1 -> 10.1.1.4)
Tx packets: 1418948
Rx packets: 1418849
Lost Packets (Tx-Rx): 99
Throughput: 6.46399 Mbps
Avg Delay: 0.0863497 s
```

```
Flow 2 (10.1.1.4 -> 10.1.1.1)
Tx packets: 67233
Rx packets: 67228
Lost Packets (Tx-Rx): 4
Throughput: 0.30622 Mbps
Avg Delay: 0.057621 s
```

```
Flow 3 (10.1.1.2 -> 10.1.1.4)
Tx packets: 1313153
Rx packets: 1312477
Lost Packets (Tx-Rx): 675
Throughput: 5.99557 Mbps
Avg Delay: 0.116566 s
```

```
Flow 4 (10.1.1.4 -> 10.1.1.2)
Tx packets: 78079
Rx packets: 78070
Lost Packets (Tx-Rx): 8
Throughput: 0.356558 Mbps
Avg Delay: 0.0643755 s
```

```
Flow 5 (10.1.1.3 -> 10.1.1.4)
Tx packets: 183116
Rx packets: 183116
Lost Packets (Tx-Rx): 0
Throughput: 0.838278 Mbps
Avg Delay: 0.0016148 s
```

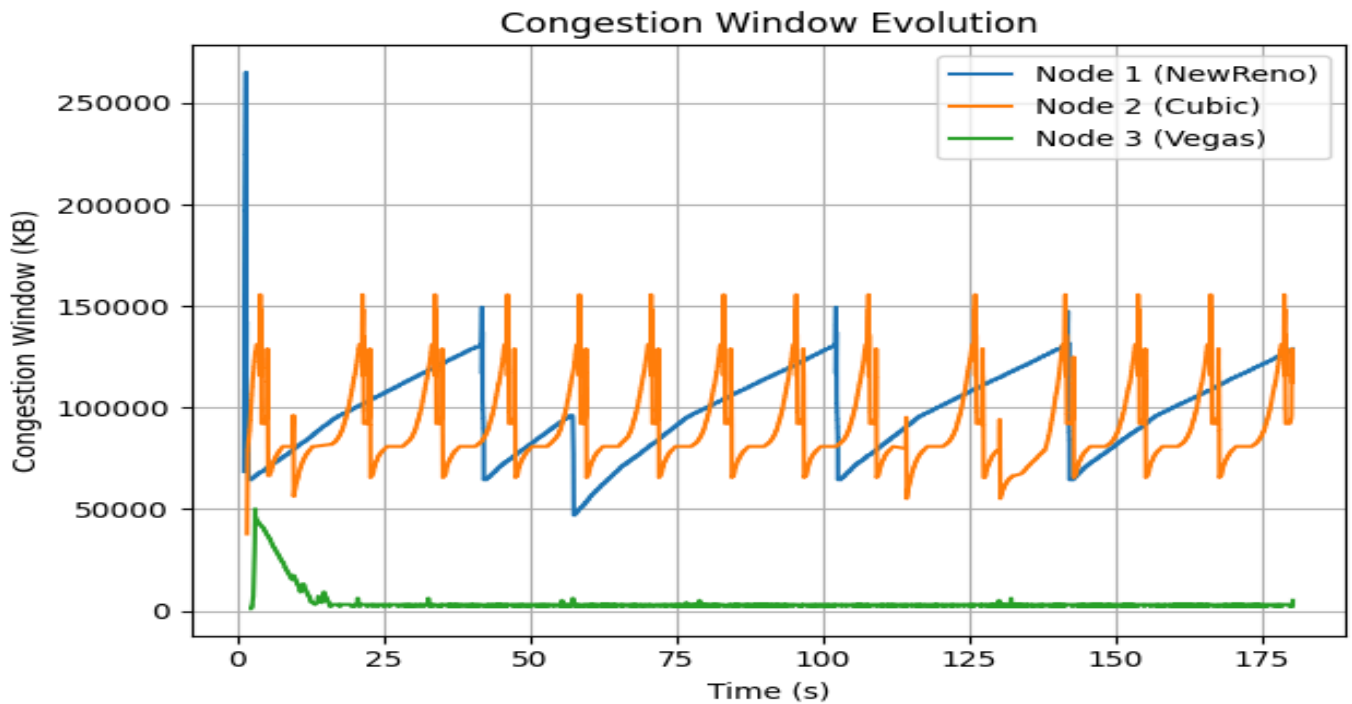
```
Flow 6 (10.1.1.4 -> 10.1.1.3)
Tx packets: 8100
Rx packets: 8100
Lost Packets (Tx-Rx): 0
Throughput: 0.0371099 Mbps
Avg Delay: 0.0409576 s
```

Flow	Direction	Tx Packets	Rx Packets	Lost Packets	Throughput (Mbps)	Avg Delay (s)
1	10.1.1.1 → 10.1.1.4	943,272	943,193	79	6.46	0.0812
2	10.1.1.4 → 10.1.1.1	45,173	45,173	0	0.31	0.0586
3	10.1.1.2 → 10.1.1.4	882,321	881,879	441	6.07	0.1114
4	10.1.1.4 → 10.1.1.2	52,076	52,073	2	0.36	0.0654
5	10.1.1.3 → 10.1.1.4	117,696	117,696	0	0.81	0.0017
6	10.1.1.4 → 10.1.1.3	5,207	5,207	0	0.04	0.0492

Flow Statistics:

- Flow 1 (NewReno) achieved the highest throughput at 6.46 Mbps with minimal packet loss (79 packets)
- Flow 3 (Cubic) achieved the second highest throughput at 6.07 Mbps but experienced significantly higher packet loss (441 packets)
- Flow 5 (Vegas) had the lowest throughput at 0.81 Mbps but experienced no packet loss and dramatically lower delay

TCP variants Comparison:



Variant	CWND Behavior	Throughput	Delay	Stability
NewReno	Conservative increase, big drops	Moderate	Moderate	Moderate
Cubic	Aggressive increase, sharp drops	High	High (queuing)	Unstable in loss-prone links
Vegas	Early convergence, stable	Low	Low	High

Comparative Performance Analysis:

Throughput vs. Delay Tradeoff: Our results demonstrate the fundamental tradeoff between throughput and delay across TCP variants:

- NewReno achieves high throughput (6.51 Mbps) with moderate delay (0.075s)
- Cubic shows good throughput (5.85 Mbps) but highest delay (0.107s)
- Vegas delivers significantly lower throughput (1.05 Mbps) but exceptional delay performance (0.0025s)

Impact of Wireless Environment:

The PHY layer packet drops demonstrate the harsh wireless environment created for testing. This validates our simulation as a good representation of real-world challenging wireless conditions. The congestion control algorithms respond differently to these conditions:

- **Loss-based algorithms** (NewReno, Cubic) interpret all packet losses as congestion signals, leading to unnecessary window reductions in wireless environments where losses may be due to channel errors rather than congestion.
- **Delay-based algorithms** (Vegas) maintain lower window sizes preemptively, resulting in lower throughput but much higher stability and significantly reduced delay.

Based on our findings:

- **NewReno:** Best suited for applications requiring good throughput with moderate delay tolerance.
- **Cubic:** Appropriate for high-bandwidth networks where rapid recovery is important.
- **Vegas:** Ideal for delay-sensitive applications like VoIP or online gaming where consistent low latency is critical.

Functions/Variable Types Mainly Used for Simulation:

`AsciiTraceHelper asciiTraceHelper;`

→ helps create ASCII trace output files (like for cwnd tracing).

Inside main:

`NodeContainer wifiStaNodes; wifiStaNodes.Create(x);`

→ creates WiFi STA nodes.

`NodeContainer wifiApNode; wifiApNode.Create(x);` → creates AP node.

YansWifiChannelHelper channel; → configures the WiFi channel.

channel.SetPropagationDelay(...); → sets constant propagation delay model.

channel.AddPropagationLoss(...); → adds path loss models

phy.Set("TxPowerStart", DoubleValue(15.0)); → sets fixed transmit power.

phy.Set("RxNoiseFigure", DoubleValue(7.0)); → models receiver noise.

phy.Set("CcaEdThreshold", DoubleValue(-82.0));
→ sets threshold for clear channel assessment.

phy.SetErrorRateModel("ns3::YansErrorRateModel"); → sets PHY error model.

wifi.SetRemoteStationManager(...);

→ fixes data/control rates and station manager.

wifi.SetStandard(WIFI_STANDARD_80211a); → sets 802.11a WiFi standard.

mac.SetType("ns3::StaWifiMac", ...); → configures STA MAC layer

staDevices = wifi.Install(phy, mac, wifiStaNodes);
→ installs WiFi stack on STA nodes.

apMac.SetType("ns3::ApWifiMac", ...); → configures AP MAC layer.

apDevice = wifi.Install(phy, apMac, wifiApNode);
→ installs WiFi stack on the AP node.

mobility.SetPositionAllocator(...); → allocates positions for STA nodes.

mobility.SetMobilityModel(...); → assigns mobility model to STA nodes.

mobility.Install(wifiStaNodes); → installs mobility on STA nodes.

mobility.SetMobilityModel("ns3::ConstantPositionMobilityModel");
→ makes AP node stationary.

mobility.Install(wifiApNode); → applies mobility model to AP.

InternetStackHelper stack; stack.Install(...); → installs internet protocol stack.

staInterfaces = address.Assign(staDevices); apInterfaces = address.Assign(apDevice); → assigns IP addresses to devices.

Ipv4GlobalRoutingHelper::PopulateRoutingTables(); → fills in routing tables automatically.

PacketSinkHelper sinkHelper(...); → configures TCP sink (receiver) at AP.

sinkApp = sinkHelper.Install(wifiApNode.Get(0)); → installs the sink on AP node.

sinkApp.Start(...); sinkApp.Stop(...); → sets start and stop times for sink.

OnOffHelper client(...); client.SetAttribute(...);
→ configures TCP OnOff application (traffic generator).

clientApp = client.Install(...); → installs the application on STA node.

onoff->TraceConnectWithoutContext("Socket", ...);
→ connects callback for TCP socket creation.

Simulator::Schedule(...); → schedules congestion window tracing for that node.

Simulator::Schedule(Seconds(1.1), &ThroughputMonitor, ...);
→ schedules throughput monitoring to start shortly after traffic begins.

FlowMonitorHelper flowmon; monitor = flowmon.InstallAll();
→ sets up FlowMonitor for stats collection.

Simulator::Stop(...); Simulator::Run(); → stops and runs the simulation.

monitor->CheckForLostPackets(); → checks for any lost packets post-simulation.

The Entire Workflow of the code is as follows:

1. **Network Initialization:** It creates 3 wireless client nodes and 1 access point using NS-3's WiFi module with combined LogDistance/Nakagami propagation models to induce high packet loss.

2. **TCP Configuration:** Assigns different congestion algorithms (NewReno/Cubic/Vegas) to each node, tuning Vegas parameters ($\alpha=2$, $\beta=4$) for aggressive delay sensitivity.
3. **Traffic Setup:** Uses OnOffApplications with 20Mbps constant TCP traffic between stations and AP, staggered by 0.5s starts to avoid synchronization.
4. **Mobility Simulation:** Implements random 2D node movement within 30m bounds while keeping the AP stationary.
5. **Real-time Monitoring:** Tracks congestion window changes at 1ms resolution, PHY-layer packet drops, and calculates throughput every second.
6. **Execution & Data Collection:** Runs 180s simulation with FlowMonitor capturing per-flow statistics (delay/loss/throughput) and exports XML results.
7. **Post-processing:** Generates congestion window evolution plots and aggregates PHY-layer drop counts to quantify wireless channel harshness.

The code systematically compares TCP behaviors by combining controlled wireless degradation (through propagation models) with granular metric collection, enabling direct observation of protocol trade-offs.

Conclusion:

This project highlights the fundamental trade-offs between delay minimization and throughput maximization in TCP congestion control algorithms in wireless networks. Cubic and NewReno, being loss-based algorithms, favor bandwidth usage at the expense of higher latency, whereas Vegas's delay-oriented design trades off throughput to keep low queuing delays. The study highlights

the fact that wireless environments, with their intermittent packet losses, disrupt conventional congestion control models, with a preference towards algorithms that can respond to both loss and delay indications. They highlight the importance of context-aware protocol selection, where the needs of the application (throughput versus latency) and the network conditions determine the best choice. The future could indeed be in hybrid methods which dynamically balance throughput, latency, and reliability by adapting to real-time network conditions.

Thank You